Experiment 6.3: Manual Triage of Findings

Issue Selection

All 14 issues reported by Infer are of the type "Thread Safety Violation". Since there is only one issue category present, I have selected two issues from this category for analysis:

- **Issue A**: Issue #0 (most frequent category: Thread Safety Violation)
- **Issue B**: Issue #2 (same category, different issue for comparison)

Issue A: Thread Safety Violation in JsonStreamParser

Issue Type and Name

- **Type**: Thread Safety Violation
- **Name**: Unprotected write. Non-private method `JsonStreamParser.next()` indirectly writes to field `this.parser.limit` outside of synchronization.

File and Method Name

- **File**: `src/main/java/com/google/gson/JsonStreamParser.java`
- **Method**: `next()` (indirectly through `Streams.parse(parser)`)

Relevant Code Snippet

```
public final class JsonStreamParser implements Iterator<JsonElement> {
    private final JsonReader parser;
    private final Object lock;

    // ... constructor ...

    @Override
    public JsonElement next() {
        // ... hasNext check ...
        try {
            return Streams.parse(parser);  // Line 89: Indirect write to parser.limit
        } catch (StackOverflowError | OutOfMemoryError e) {
            throw new JsonParseException("Failed parsing JSON source to Json", e);
        }
    }
}
```

Excerpt of the Infer Trace

```
src/main/java/com/google/gson/JsonStreamParser.java:51: warning: Thread Safety Violation
  Unprotected write. Non-private method `JsonStreamParser.next()` indirectly writes to field `this.parser.limit`
outside of synchronization.
 Reporting because another access to the same memory occurs on a background thread, although this access may not.
  49.    * @since 1.4
  50.    */
  51. > public final class JsonStreamParser implements Iterator<JsonElement> {
  52.     private final JsonReader parser;
  53.     private final Object lock;
```

Execution Path Evidence Table

Trace Step	File	Line	Explanation
Value introduced	`src/main/java/com/google/gson/JsonStreamParser.java`	52	The `parser` field (JsonReader) is introduced as a private final field, shared across method calls. Value is declared in the constructor using an external call to `src/main/java/com/google/gson/stream/JsonReader.java` which creates a new instance that reads a JSON-encoded stream.
Conditional branch	N/A	N/A	No conditional branch in the reported trace; the issue occurs on method entry.

Trace Step	File	Line	Explanation
Risky state appears	src/main/java/com/google/gson/JsonStreamParser.java	89	The next() method in JsonStreamParser calls Streams.parse(parser), which performs JSON parsing operations on the JsonReader instance (parser). During parsing, the reader may need to read more data from the underlying stream, triggering buffer management that modifies the limit field (an internal buffer position/length tracker in JsonReader). This happens indirectly through methods like fillBuffer(), where limit is updated to reflect new data read into the buffer.
Failure location	src/main/java/com/google/gson/stream/JsonReader.java	1492	The unprotected write to parser.limit occurs here, potentially racing with other threads. RacerD/Infer is flagging an unprotected write through JsonStreamParser.next() to parser.limit via Streams.parse(parser) → JsonReader methods like peek()/doPeek() → nextNonWhitespace() → fillBuffer().

Trace Understanding Questions

T1. First Feasible Failure Point

- **File and line number**: src/main/java/com/google/gson/JsonStreamParser.java, line 89 (`return Streams.parse(parser);`)
- **Explanation**: This is the earliest point where the failure becomes possible. The `Streams.parse(parser)` call modifies the `parser.limit` field, and since the `next()` method is not synchronized, concurrent calls to `next()` or other methods accessing `parser` could lead to a race condition. The class has a `lock` field but does not use it, making this the first risky operation.

T2. Infer Assumption

- **Assumption**: Infer assumes that the `JsonStreamParser` instance can be accessed concurrently by multiple threads, and that there is another thread performing operations on the same `parser` object that access or modify `parser.limit`.
- **Explanation and code location**: This assumption is based on the presence of the `parser` field (line 52) being a shared mutable object. The code does not synchronize access to `parser`, and Infer models potential concurrent execution paths where one thread calls `next()` while another accesses `parser.limit` elsewhere or that multiple threads are simultaneuosly executing `next()`.

T3. Refactoring Robustness

- **Semgrep**: Semgrep (pattern-matching based) would likely NOT detect the issue if its rules are simple and only pattern-based. If Semgrep was customized to define rules with `threadid()` or a concurrent rule package was used, it could detect the issue. However, if the rules are tied to specific method names or exact code patterns, a refactor like renaming `next()` to `getNextElement()` might evade detection. Meaning semgrep is not path-sensitive unless custom pattern engine.
- **Infer**: If the change is a behavior-preserving refactoring Infer *would still report the issue* because it performs path-sensitive analysis. The core problem is the unsynchronized access to shared state (`parser.limit`), so renaming the method or extracting variables wouldn't change the execution path or eliminate the race condition. However: Assuming that thread issues are checked solely by RaceD, Infer uses RaceD to detect concurrency issues, and as the documentation states, certain limitations have been listed: (1) It looks for races involving syntactically identical access paths, and misses races due to aliasing (2) It misses races that arise from a locally declared object escaping its scope (3) It uses a boolean locks abstraction, and so misses races where two accesses are mistakenly protected by different locks (4) It assumes a deep ownership model, which misses races where local objects refer to or contain non-owned objects. (5) It avoids reasoning about weak memory and Java's volatile keyword Additionally, RaceD is triggered via: (1) Explicitly annotating a class/method with @ThreadSafe and (2) using a lock via the synchronized keyword. In both cases, RacerD will look for concurrency issues in the code containing the signal and all of its dependencies. In particular, it will report races between any non-private methods of the same class that can perform conflicting accesses. Annotating a class/interface with @ThreadSafe also triggers checking for all of the subclasses of the class/implementations of the interface. If the programmer would remove the lock/sync keywords, remove any @ThreadSafe flags, *RaceD might not detect the issue*.
- **Reasoning**: Semgrep relies on syntactic patterns, while Infer traces data flow and synchronization, making it more robust to superficial changes but sensitive to actual concurrency issues.

Classification

- **Classification**: Likely false positive
- **Justification**: Some of Gson's classes are explicitly documented as not thread-safe or conditionally thread-safe: In `JsonStreamParser` the authors write:

```
/*
 * <p>This class is conditionally thread-safe (see Item 70, Effective Java second edition). To
 * properly use this class across multiple threads, you will need to add some external
 * synchronization. For example:
 *
 * <pre>
 * JsonStreamParser parser = new JsonStreamParser("['first'] {'second':10} 'third'");
 * JsonElement element;
 * synchronized (parser) {  // synchronize on an object shared by threads
 *   if (parser.hasNext()) {
 *     element = parser.next();
 *   }
```

```
 * }
 * </pre>.
```

and in `JsonReader` they write:

```
/*
 * <p>Each {@code JsonReader} may be used to read a single JSON stream. Instances of this class are
 * not thread safe.
```

The `JsonStreamParser` class is designed for single-threaded use, and the presence of an unused `lock` field suggests incomplete thread-safety implementation rather than an intended defect. In practice, users are expected to synchronize externally or use separate instances per thread.

Fix Validation

Since this is classified as a false positive, no fix is necessary. However, for demonstration, I added synchronization using the existing `lock` field: `synchronized (lock) { return Streams.parse(parser); }`. This modified the `next()` method in `JsonStreamParser.java` to synchronize on `lock`.

```
  @Override
  public JsonElement next() throws JsonParseException {
    synchronized (lock) {
      if (!hasNext()) {
        throw new NoSuchElementException();
      }

      try {
        return Streams.parse(parser);
      } catch (StackOverflowError | OutOfMemoryError e) {
        throw new JsonParseException("Failed parsing JSON source to Json", e);
      }
    }
  }
```

After re-running `infer run -- mvn compile`, the issue disappears from the output for this specific location, as the write is now protected (see `infer-validation/report.txt`).

- **Report**: The fix eliminates the reported issue, but this may not be the intended behavior since Gson is not thread-safe overall. Note: both #0 and #1 from `infer-out/report.txt` dissappeared as they were pointing to the same issue.

Issue B: Thread Safety Violation in DefaultDateTypeAdapter

Issue Type and Name

- **Type**: Thread Safety Violation
- **Name**: Read/Write race. Non-private method `DefaultDateTypeAdapter.write(...)` indirectly reads without synchronization from `out.deferredName`, which races with the write in method `DefaultDateTypeAdapter.write(...)`.

File and Method Name

- **File**: `src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java`
- **Method**: `write(JsonWriter out, Date value)`

Relevant Code Snippet

```
public void write(JsonWriter out, Date value) throws IOException {
    if (value == null) {
        out.nullValue();  // Line 142: Indirect read/write to out.deferredName
        return;
    }
    // ... rest of method ...
    out.value(dateFormatAsString);  // Line 152: Another access
}
```

Excerpt of the Infer Trace

```
src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java:142: warning: Thread Safety Violation
  Read/Write race. Non-private method `DefaultDateTypeAdapter.write(...)` indirectly reads without synchronization
from `out.deferredName`, which races with the write in method `DefaultDateTypeAdapter.write(...)`.
 Reporting because this access may occur on a background thread.
  140.     public void write(JsonWriter out, Date value) throws IOException {
  141.       if (value == null) {
```

```
142. >       out.nullValue();
143.         return;
144.       }
```

Execution Path Evidence Table

Trace Step	File	Line	Explanation
Value introduced	src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java	140	The `out` parameter (JsonWriter) is introduced, containing the `deferredName` field.
Conditional branch	src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java	141	The `if (value == null)` branch leads to the risky *simulatenous* access.
Risky state appears	src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java	142	The call to `out.nullValue()` accesses `out.deferredName` without synchronization.
Failure location	src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java	142	The race occurs here due to unsynchronized read/write on `deferredName`.

Trace Understanding Questions

T1. First Feasible Failure Point

- **File and line number**: `src/main/java/com/google/gson/internal/bind/DefaultDateTypeAdapter.java`, line 142 (`out.nullValue();`)
- **Explanation**: This is the earliest point where the race becomes possible. The `out.nullValue()` method accesses `out.deferredName`, and since `write()` is not synchronized, concurrent calls to `write()` on the same `JsonWriter` could race on `deferredName`.

T2. Infer Assumption

- **Assumption**: Infer assumes that the `DefaultDateTypeAdapter.write()` method can be called concurrently on the same `JsonWriter` instance, leading to races on `out.deferredName`. This contradicts the author's instructions previously mentioned in Issue A's classification.
- **Explanation and code location**: This is based on the `out` parameter (line 140) being a mutable object passed in, and the lack of synchronization in the method. Infer models scenarios where multiple threads serialize dates using the same writer. However, authors have noted that instances of `JsonWriter` may be used to write to a single JSON stream and are NOT thread-safe.

T3. Refactoring Robustness

- **Semgrep**: Semgrep would likely still detect the issue if its rules target method calls like `out.nullValue()` without synchronization. Again, if semgrep is customized with the appropriate semantic rules to handle concurrency detection in great depth, then it could detect the concurrency issue, since this issue is quite trivial. However, semgrep has its limitations in detecting the more complex, cross-function, or cross-file timing and ordering issues like generic race conditions or infinite loops.
- **Infer**: Infer would still report the issue, as the data flow and lack of synchronization remain the same. Path-sensitive analysis focuses on the actual concurrency problem, not variable names.
- **Reasoning**: Semgrep's pattern matching is more brittle to code changes, while Infer's deeper analysis makes it resilient to refactoring that doesn't address the root cause.

Classification

- **Classification**: Likely false positive
- **Justification**: Similar to Issue A, Gson type adapters are not designed for concurrent use. The `JsonWriter` is typically used in a single-threaded context, and synchronizing every adapter method would be impractical. The race is theoretical and not a real defect in intended usage.

Summary

Both issues are classified as likely false positives due to Gson's non-thread-safe design. The analysis highlights Infer's strength in detecting potential concurrency issues, even if they are not practical defects. No fixes were applied beyond demonstration for Issue A, as the issues do not represent real bugs in the codebase.